

Interactive Smart Contracts for Trustworthy and Transparent Online Content Consumption Tracking

Shubham Joshi, Dillon Davidson, Zuobin Xiong, and Junggab Son*

Abstract—The rapid evolution of the digital landscape has made content creation and distribution more accessible than ever. However, ensuring the integrity and accuracy of content consumption and download tracking remains a significant challenge. Centralized tracking systems often lack transparency and are susceptible to security breaches, raising concerns for both content creators and service providers. To address these issues, we propose a blockchain-based solution that leverages smart contracts for tamper-proof content tracking. Our approach introduces two secure and interactive smart contracts: an Elliptic Curve Cryptography-based Proxy Re-Encryption smart contract (ECC-PRE-SC) and a Counting smart contract (Count-SC). The ECC-PRE-SC facilitates secure re-encryption, ensuring that only authorized entities can access the encrypted content key. Meanwhile, the Count-SC maintains an immutable and verifiable record of content consumption and downloads. Content Owners (COs) deploy the ECC-PRE-SC, embedding a list of authorized service providers' wallet addresses to enforce access control and prevent unauthorized modifications to download counts. Each ECC-PRE-SC instance is linked to a dedicated Count-SC, ensuring precise and independent tracking for specific content. Our security analysis verifies the system's robustness against potential threats, while its successful deployment on the Sepolia Testnet demonstrates its feasibility. This research enhances trust and transparency in digital content distribution, providing significant benefits to content creators, service providers, and end users.

Index Terms—Content Consumption Tracking, Interactive Smart Contract, Online Content Sharing, Trustworthiness, Blockchain.

I. INTRODUCTION

CONTENT creation is evolving rapidly in today's fast-paced digital world. The widespread availability of the Internet has made it easier than ever to share content and reach larger audiences [2]. However, one significant challenge for Content Owners (COs) is ensuring secure distribution [3]. To address this, cloud-sharing platforms, referred to as Service Providers (SPs), have emerged, helping COs to grow their user base while maintaining security. This ecosystem benefits both COs and SPs, as the success and profitability of content often depend on its popularity, typically measured by the number of times it is consumed or downloaded [4]. Therefore, accurate and tamper-proof records of content consumption are essential for both parties.

Numerous challenges still persist in the traditional management of digital rights for online media [5]. A critical issue lies

in ensuring the accuracy and tracking of the count for content consumption or downloads. In most cases, SPs manage these count through proprietary software, which they can modify at will. This creates a trust deficit between COs and SPs, as their profitability hinges on transparent and accurate reporting of content consumption [6].

Many Digital Rights Management (DRM) systems are currently used to manage content rights, facilitating the transfer of these rights from COs to users via intermediaries. These DRM systems are typically centralized, controlled, and operated by entities, often SPs, focused on distributing content to a broader audience. A key advantage of this centralized approach is its ability to protect COs by safeguarding their intellectual property [7]. However, there are risks involved, as third parties or other entities may violate established rules and engage in harmful activities, such as distributing content to unauthorized users, which undermines the interests of COs.

Moreover, if a third-party server is compromised, it can result in sensitive digital content information being leaked. Disruptions to server access can also impact all users who depend on the service, highlighting the vulnerabilities of centralized DRM systems. The centralized nature of current DRM systems makes them susceptible to single points of failure and cyberattacks. They also lack the tamper resistance and distributed trust offered by decentralized architectures, such as blockchains [8]. Furthermore, managing content rights and usage policies across multiple stakeholders in the content distribution ecosystem is hindered by limited flexibility and transparency [9], [10].

To address the challenge of maintaining integrity and ensuring a tamper-proof count of content consumption or downloads, we propose a system that leverages two secure smart contracts: an Elliptic Curve Cryptography-based Proxy Re-Encryption Smart Contract (ECC-PRE-SC) and a Counting Smart Contract (Count-SC). The ECC-PRE-SC manages the re-encryption process as part of granting access to content and also acts as a verifier for the Zero Knowledge Proof process [11], while the Count-SC records the number of times the ECC-PRE-SC is executed successfully.

The CO deploys the ECC-PRE-SC, embedding the wallet addresses of authorized SPs and the encrypted content key to facilitate re-encryption operations. When the constructor of ECC-PRE-SC is invoked, it creates a new instance of the Count-SC within the ECC-PRE-SC contract. This triggers the deployment of the Count-SC, with the ECC-PRE-SC's address designated as the sole authorized entity allowed to execute the increment function in the Count-SC. Furthermore, This design ensures that the count can only be modified through

S. Joshi, D. Davidson, Z. Xiong, and J. Son are with Department of Computer Science, University of Nevada, Las Vegas, Las Vegas, Nevada, USA. Email: {shubham.joshi, junggab.son@unlv.edu}

A preliminary version of this work was presented at SecureComm 2024 and was published in its proceeding [1]

*Corresponding Author

the increment function, which must be called by an authorized entity.

The proposed system effectively addresses the challenges of ensuring the integrity of the content consumption count. By leveraging smart contracts, we provide a solution that enables tamper-proof tracking and recording of content downloads or usage. This approach significantly improves trust and transparency between *COs* and *SPs* throughout the content distribution process. Furthermore, the proposed scheme contributes to the advancement of decentralized architectures, such as blockchain, providing a more robust alternative to traditional centralized DRM systems. The outcomes of our research benefit not only *COs* and *SPs* but also users who rely on secure and transparent platforms for content distribution.

The main contributions of this paper can be summarized as follows:

- In current research, as far as we are aware, there is a noticeable lack of research on the accuracy of content consumption or download counts. Recognizing this gap led to the development of the solution presented in this paper.
- An interactive smart contract architecture has been proposed that ensures the integrity of content consumption count. One contract leverages the efficiency of Elliptic Curve Cryptography-based Proxy Re-encryption Scheme for re-encryption, while the other provides the integrity of the count for how many times content has been consumed or downloaded.
- A security proof based on a defined threat model has been provided that shows that our system is secure. It is resilient to adversaries trying to modify the count in the Count-SC or gain unauthorized access to content keys through re-encrypted content keys on the blockchain.
- The feasibility of the proposed architecture is demonstrated via implementation on Sepolia Testnet. The implementation also shows consistent gas consumption during re-encryption, even with varying content key lengths in the smart contract.
- An efficient version of the smart contract was developed, which improves the usage of gas by approximately 63% from our previous version.

The remainder of this paper is organized as follows. Section II reviews related work on encryption schemes, smart contract-based DRM, and cloud content sharing. Section III covers preliminaries, laying the foundation for our system. Section IV defines the problem through the system model, threat model, and design goals. Our proposed scheme, including two key smart contracts, is presented in Section V. Section VI validates the correctness of the scheme, while Section VII provides a formal security analysis. Section VIII discusses implementation and evaluation, and Section IX concludes with a summary of our contributions.

II. RELATED WORK

This section explores recent research trends in secure content sharing within cloud environments. We specifically examine studies focused on encryption schemes, cloud content

sharing mechanisms, and the application of smart contracts within DRM systems.

A. Cloud Content Sharing

Several studies address the challenges of secure cloud content sharing. Zhang et al. introduced CREAMS, a scheme prioritizing copyright protection and data privacy within media sharing contexts [12]. Their design combines additive homomorphic proxy re-encryption with asymmetric fingerprinting techniques to address the problem of data privacy leakage, infringement on the data owner's copyright.

Feng et al. proposed a proxy re-encryption scheme combining both attribute-based encryption with identity-based encryption [13]. Integrated with blockchain technology to solve issues with data tampering and trusted third parties, their scheme is an upgrade as it is efficient and verifiable so that dishonest proxy nodes cannot leak sensitive data.

Liu et al. proposed a scheme that resolved common issues related to certificate management and key escrow in traditional proxy re-encryption schemes and improved the computational efficiency by eliminating the reliance on bilinear pairs. [14]. Their scheme uses elliptic curve groups combined with certificated-based encryption and identity-based proxy re-encryption to greatly improve efficiency, guarantee non-transferability, and enhance functionality by solving the problems of certificate revocation and key escrow.

B. DRM based on Smart Contracts

The potential of blockchain and smart contracts in DRM systems is evident in recent research. One study sought to overcome the limitations of traditional centralized DRM frameworks by using smart contracts to facilitate reliable copyright transactions [15].

Another significant work proposes a distributed media transaction framework that merges digital watermarking with a scalable blockchain model for DRM [16]. This framework prioritizes secure authorized access to online content, protection of offline content copyright through watermarking, and mechanisms for copyright reclamation in case of leakage.

Gao et al. proposed a blockchain-based DRM scheme designed to improve content confidentiality, enhance fairness in copyright transactions, and streamline management for copyright holders [17]. Their scheme leverages a new Multi-Authority Ciphertext-Policy Attribute-Based Encryption (MA-CPABE) system for secure encryption and integrates proxy re-encryption for efficient copyright transfers. Smart contracts on Ethereum facilitate secure and transparent transactions of decryption keys between rights owners and content requesters.

C. Encryption Schemes for Secure Data Sharing

In 2016, Thangam and Chadrasekaran addressed shortcomings in earlier Proxy Re-Encryption (PRE) schemes, such as bidirectionality, collusion-resistance, and re-encryption key generation [18]. They proposed two new schemes, CTR1 and CTR2, an elliptic curve based PRE schemes. Notably, CTR2 builds upon the work of Ateniese et al., whose pioneering PRE

scheme eliminated the need for pre-shared elements, offering unidirectional and collusion-resistant features [19].

Agyekum et al. (2022) focused on secure data sharing within IoT environments. They combined identity-based encryption with PRE to enable robust access control mechanisms [20]. More recently, Keshta et al. explored the integration of blockchain technology with PRE [21]. Their scheme utilizes the SM2 algorithm within a blockchain-based data-sharing framework, emphasizing data security and privacy enhancements.

Similarly, one of the studies by Shabisha et al. explored the integration of Elliptic Curve Cryptography (ECC) with implicit certificate-based signcryption and PRE techniques for secure and efficient data storage within cloud environments [22]. This research focuses on optimizing these cryptographic primitives for the cloud setting, addressing concerns like data security against a potentially curious cloud provider.

III. PRELIMINARIES

This section introduces important preliminaries that are the building blocks of our proposed system.

A. Blockchain and Smart Contracts

Blockchain is a unique data structure composed of sequentially linked data blocks in a chain-like format. Operating within a peer-to-peer network and governed by a reliable consensus mechanism, it ensures data integrity and prevents unauthorized alterations, making it secure, traceable, decentralized, and trustless. As a distributed ledger, verified transactions are added to the blockchain with timestamps, creating a continuously growing data repository. Blockchains are generally categorized into three types: public, consortium (alliance), and private. Notable characteristics of blockchain include persistence, where once a block is added, it becomes immutable, and public verifiability, which allows anyone to verify the integrity of the data. These features make the blockchain an attractive foundation for developing applications that do not rely on trusted intermediaries [23] [24].

Smart contracts are essentially coded programs distributed across a decentralized blockchain network. They facilitate transactions between parties, even if they are anonymous or lack mutual trust, eliminating the need for a central authority [25]. Various blockchain platforms, including Bitcoin [26], Ethereum [27], Hyperledger Fabric [28], Stellar [29], iOlive [30], and Lisk [30], are available for smart contract development. Nevertheless, Ethereum [31] [32] stands out as the most widely adopted platform, attributed to its stability and secure operation.

B. Hardness Assumption

Our scheme's security is based on the following defined hard problems. First, we introduce the concept of elliptic curves. An elliptic curve $E(\mathbb{F}_q)$ is defined over a finite field $\mathbb{F}_q$. It has a base point generator $P \in E(\mathbb{F}_q)$ of prime order q . Together with the infinite point, all points on $E(\mathbb{F}_q)$ form an additive group G . Given this, we consider a Probabilistic Polynomial-Time (PPT) adversary $\mathcal{A}$ and define the following hard problems:

1) *Discrete Logarithm Problem (DLP)*: : Let $(x, pk) \leftarrow \text{Gen}(1^\lambda, P)$ be a function that takes as inputs a security parameter 1^λ and the base point P , then outputs $x \in \mathbb{Z}_n^*$ and $pk = x \cdot P \in E(\mathbb{F}_q)$. Given $1^\lambda, pk$, and P as inputs, the probability that $\mathcal{A}$ outputs x' such that $x' = x$ should be negligible.

$$\Pr[x' \leftarrow \mathcal{A}(1^\lambda, pk, P) : x' = x] \leq \text{negl}(\lambda)$$

2) *Computational Diffie-Hellman (CDH) Problem*: : Let G be a large prime elliptic curve group of order q and P a generator of the group G . Then, the CDH problem on group G is defined as follows: Given $P, aP, bP \in G$, compute $abP = P \in G$ for any $a, b \in \mathbb{Z}_q^*$. The probability that any probabilistic polynomial-time (PPT) algorithm, denoted as $\mathcal{A}_{CDH}$, that can effectively solve the Computational Diffie-Hellman (CDH) problem is negligible:

$$\Pr[\mathcal{A}_{CDH}(G, q, P, aP, bP) = abP] \leq \text{negl}(\lambda)$$

C. Elliptic Curve based Proxy Re-encryption

Proxy Re-Encryption (PRE) schemes enable proxies to transform an encrypted message intended for one recipient so that it can be decrypted by a different recipient. For example, Alice can authorize a proxy to convert her encrypted emails into a format that Bob can decrypt without disclosing her secret key to him. This eliminates the need for Alice to trust Bob with her secret key. The algorithms needed for the PRE schemes are briefly outlined below:

- **Setup**(1^λ): This takes (λ) as security parameter and generates public parameters $params$.
- **KeyGenerate**($params$): This takes parameters, $params$, as input and returns the private key, sk_i , and corresponding public key, $pk_i = (pk_{i1}, pk_{i2})$.
- **Encrypt**($k_i, id_{CO}, pk_{CO}, params$): This takes the content key, k_i , identity of the CO, id_{CO} , public key of the CO, pk_{CO} , and public parameters, $params$, as input and returns the encrypted content, $C = (C_1, C_2, C_3, C_4, C_5)$.
- **ReKeyGenerate**($c, params, id_{CO}, id_{U_i}, pk_{CO}, pk_{U_i}$): This takes a ephemeral randomness, c , public parameters, $params$, identity of the CO, id_{CO} , public key of the CO, pk_{CO} , identity of the User, id_{U_i} , and public key of the User, pk_{U_i} , as input and returns a re-encryption key, $rk_{A \rightarrow B} = (rk_1, rk_2, rk_3)$.
- **Re-Encrypt**($C, rk_{A \rightarrow B}, params$): This takes the encrypted content, C , re-encryption key, $rk_{CO \rightarrow U}$, and public parameters, $params$, as input, and it returns the re-encrypted content, $C' = (C'_1, C'_2, C'_3, C'_4)$.
- **Decrypt**($C, id_{CO}, sk_{CO}, params$): This takes the encrypted content, C , identity of the CO, id_{CO} , private key of the CO, sk_{CO} , and public parameters, $params$, and it returns the decrypted content, k_i .
- **Re-Decrypt**($C', id_{CO}, pk_{CO}, id_{U_i}, sk_{U_i}, params$): This takes the re-encrypted content, C' , identity of the CO, id_{CO} , public key of the CO, pk_{CO} , identity of the User, id_{U_i} , private key of the User, sk_{U_i} , and public parameters, $params$, as input and returns the decrypted content, k_i .

D. Keccak256 Hash Function Library

Keccak256 [33], a specific instance of the Keccak family of cryptographic hash functions, is designed to produce a 256-bit hash value. It utilizes the sponge construction and the underlying permutation Keccak-f. The mathematical definition of the Keccak256 hash function can be outlined as follows:

- **State Representation:** The state S is represented as a 5×5 array of 64-bit lanes, resulting in a total state size of 1600 bits (or 200 bytes): $S = (S_{0,0}, S_{0,1}, S_{0,2}, S_{0,3}, S_{0,4}, S_{1,0}, S_{1,1}, S_{1,2}, S_{1,3}, S_{1,4}, S_{2,0}, S_{2,1}, S_{2,2}, S_{2,3}, S_{2,4}, S_{3,0}, S_{3,1}, S_{3,2}, S_{3,3}, S_{3,4}, S_{4,0}, S_{4,1}, S_{4,2}, S_{4,3}, S_{4,4})$
- **Sponge Construction:** Keccak256 operates using the sponge construction, which consists of two main phases: absorbing and squeezing.
- **Absorbing Phase:** Input data M is padded to ensure it fits the sponge's requirements. The padding rule for Keccak is defined such that the last byte of the input is set to 1 followed by enough 0 bits to fill the block, and the last bit of the last byte is set to 1 to indicate the end of the message. The padded message is absorbed into the state by XORing it with the state and applying the permutation Keccak-f: $S = S \oplus M_i$ for each block M_i . $S' = \text{Keccak-f}(S)$
- **Squeezing Phase** The output is generated from the state. For Keccak256, the output length is 256 bits (32 bytes). The output is taken from the state and the permutation is applied again if more output is needed: $Z = S[0] \parallel S[1] \parallel S[2] \parallel S[3] \parallel S[4]$ (first 256 bits) If more output is required, the permutation is applied again: $S' = \text{Keccak-f}(S)$
- **Final Output** The final output H of the Keccak256 hash function is the first 256 bits of the squeezed output: $H = Z[0 : 256]$
- **Mathematical Summary** The Keccak256 hash function can be summarized mathematically as: $H = \text{Keccak256}(M) = \text{Squeeze}(\text{Absorb}(M))$ where M is the input message, and the output H is a 256-bit hash value.

IV. PROBLEM DEFINITIONS

A. System Model

A system model for our proposition has been provided in Fig. 1. The model considers a scenario with a *CO* and a single blockchain network, reflecting a realistic setting where *CO* want to distribute their content through various *SPs*. It also considers multiple users who can interact with *SPs* to access content. The model consists of five entities: Trusted Third Party (*TTP*), Content Owners (*COs*), Service Providers (*SP₁, SP₂, ..., SP_n*), Blockchain Network (*BN*), and Users (*U₁, U₂, ..., U_n*).

1) *Trusted Third Part (TTP)*: The *TTP* is responsible for setting up the public parameters and generating key pairs for the *COs*, *SPs*, and users. It securely distributes the public keys and private keys to the respective parties.

2) *Content Owner (COs)*: The *COs* is the entity that produces various forms of content, such as music, videos, ebooks, and PDFs. The *CO* supplies their encrypted content to

the *SPs*, aiming to distribute it. The content may be distributed to multiple *SPs* for broader dissemination purposes. The *CO* generates the content key which is used to encrypt the content. Furthermore, the *CO* is also responsible for deploying two smart contracts to the blockchain with the encrypted content key and authorized list of wallet addresses of *SPs*. For every content, a unique content key is used for encryption, and consequently, a distinct smart contract is deployed for each.

3) *Service Providers (SPs)*: The *SPs* are entities that receive encrypted content from the *CO* and possess authorizations for distributing the contents through their designated platforms. *SPs* are also responsible for generating the re-encryption key and interacting with ECC-PRE-SC deployed in the blockchain for receiving re-encrypted content key, which will be forwarded to the User along with the encrypted content. These platforms enable users to access or download the provided content.

4) *Blockchain Network (BN)*: The *BN* serves two roles in our context. Firstly, it comprises deployed smart contracts, and secondly, it maintains an untampered ledger containing the count of any consumed or downloaded content in a decentralized network. *BN* consists of two implemented smart contracts. The first, ECC-PRE-SC, verifies the *SP* with a zero-knowledge proof and, if verified, re-encrypts the encrypted content key. The second, Count-SC, keeps track of how many times the content has been consumed or downloaded successfully.

As a decentralized network, *BN* ensures tamper-proof data for *COs* and *SPs*. The execution of the second smart contract deploys the count on the blockchain, providing tamper-proof data to *COs* and *SPs*.

5) *Users*: Users are entities that consume the contents distributed by the *SPs*. Users follow a registration process with the *SP* to validate themselves for accessing or downloading content. After registration, users need to be authenticated to access or download the content. A valid user U_i can then access or download authorized encrypted content from the *SPs* along with the re-encrypted content key, which can only be decrypted by the requested Users. During this process, the U_i needs to provide their public key to the *SP*. The user decrypts the re-encrypted content key using its private key and then decrypts the content using the content key.

B. Threat Model

The following models represent the list of attacks that may target to our system model.

- **Threat Model 1:** An adversary may try to obtain a content key by analyzing multiple re-encrypted content keys recorded on the blockchain. If the adversary browse the blockchain ledger and finds transaction hash, block hash, or address of the smart contract, they may determine the location of the transaction details and re-encrypted content key. By analyzing multiple such keys, the adversary may attempt to identify the content key, which can then be used to decrypt the content.
- **Threat Model 2:** A *CO* or *SP* may act as an adversary and attempt to maliciously manipulate the count recorded on the blockchain, seeking to change the count for personal

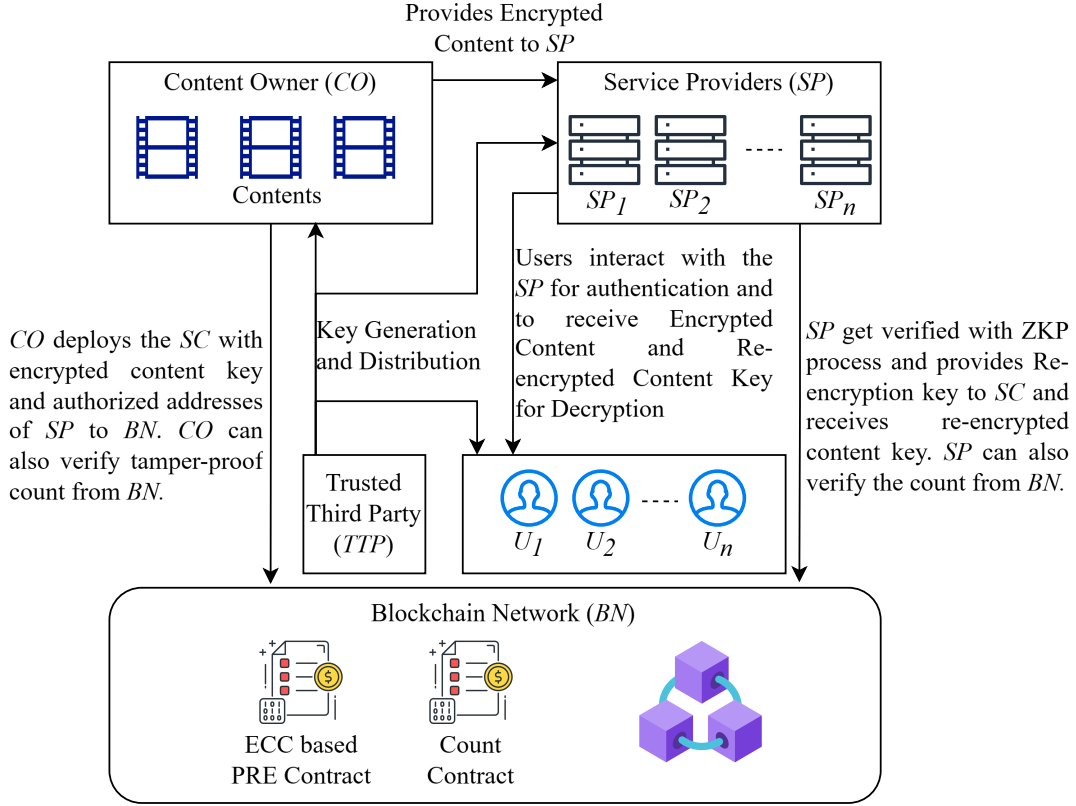


Fig. 1: System Model

gain. An unethical attempt to gain control over the Count-SC may be made in order to achieve this goal.

- Threat Model 3: A User with legitimate access to one content may attempt to leverage this access to gain unauthorized access to a different content. A User with a legitimate content key to one of the contents may maliciously try to decrypt other contents using the same key.
- Threat Model 4: The SP may act as an honest but curious entity seeking to retrieve information through the transactions that go through it. The SP serves as the intermediary for creating the re-encryption keys and forwarding the encrypted content, as well as the re-encrypted content key to the Users. The SP may attempt to obtain information regarding the content key through this process.
- Threat Model 5: An adversarial entity may attempt to impersonate an honest SP to request the re-encrypted content key from the ECC-PRE-SC or deliberately overload the ECC-PRE-SC to disrupt its functionality.

C. Design Goals

Considering the described system model and threat model, the following design goals has been established.

- The system aims to record and display the tamper-proof count of the number of content consumed or downloaded.
- The system aims to implement privacy-preserving techniques to anonymize or hide information that can otherwise be seen on the blockchain.

- The system aims to follow key independence, ensuring that there are different content keys for different contents, and they must be random during key establishment.

V. CONSTRUCTION OF THE PROPOSED SYSTEM

This section provides details of our proposed scheme and its components. It details the Elliptic Curve based Proxy Re-Encryption scheme as well as the design of the smart contracts. Table I defines notations used in this paper.

TABLE I: Description of Notations

Notation	Description
pk_{CO} & sk_{CO}	Public key & Private key of CO.
pk_{U_i} & sk_{U_i}	Public key & Private key of User.
id_{CO} & id_{U_i}	Identity of CO & Identity of User.
q	Prime order.
G	Cyclic Additive group.
P	Generating Element of G.
$rk_{CO \rightarrow U}$	Re-encryption key generated by SP and sent to the ECC-PRE-SC for re-encryption process.
C_m	Encrypted content, $E_{k_i}(m)$.
C	Encrypted Content Key by CO, $E_{pk_{CO}}(k_i)$
C'	Re-encrypted Content Key by SC1.
m	Original or unencrypted content.
k_i	Original key used to encrypt the original content.
i, o	Large Prime Numbers.
v	Secret Random Integer generated by SP in ZKP.
j	Random Number generated by the SP in ZKP.
α	Challenge generated by SP in ZKP.
γ	Proof generated by SP in ZKP.

We propose the application of an Elliptic Curve Cryptography-based Proxy Re-Encryption scheme compris-

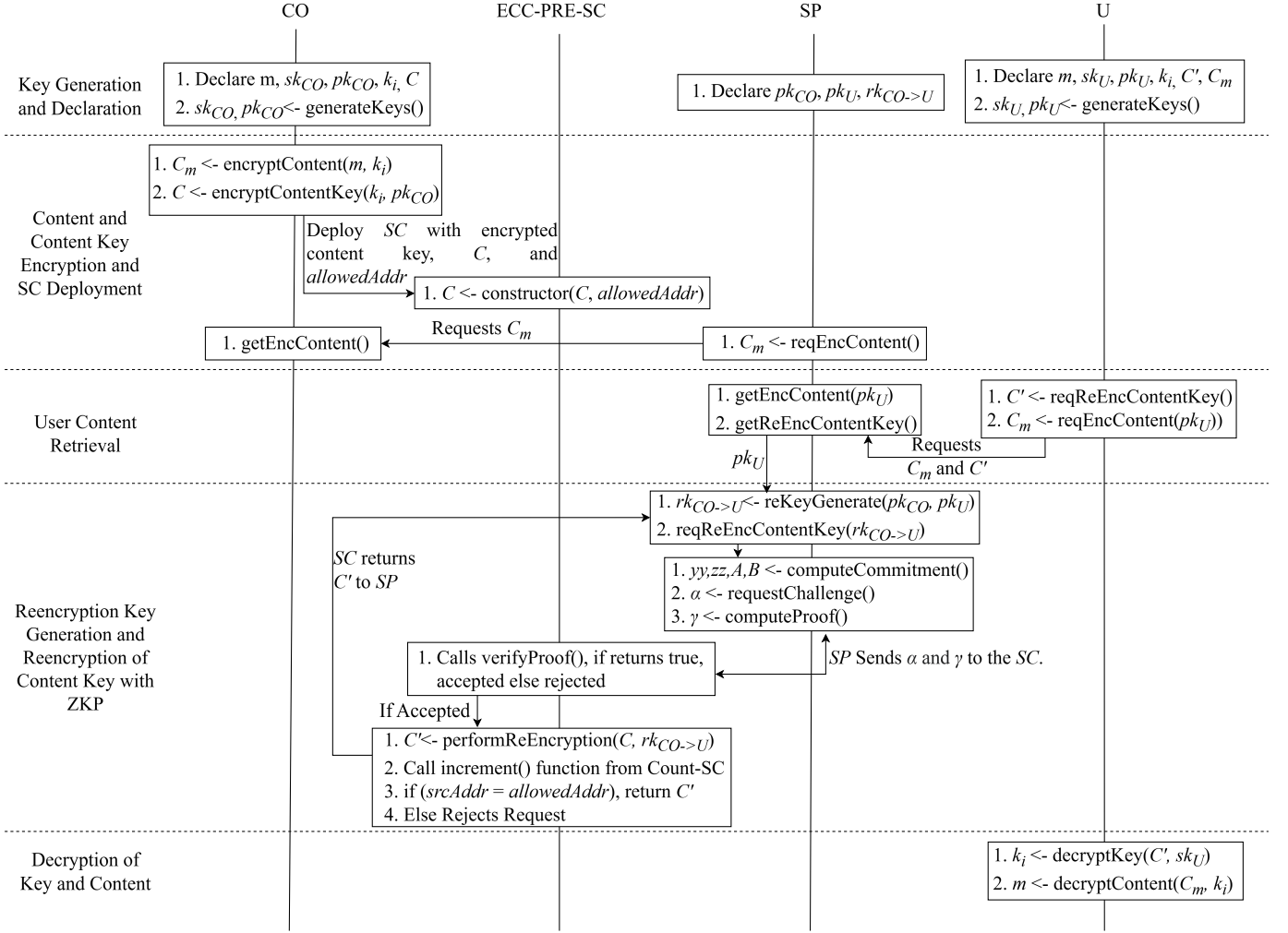


Fig. 2: Overall Process of the System reflecting ECC-PRE-SC

ing polynomial-time algorithms. The first two algorithms are essential for key generation: Setup algorithm and Key Generation Algorithm. The *TTP* sets up the public parameters using the Setup algorithm and then uses the Key Generation algorithm to generate the public and private keys for the *COs*, the *SPs*, and the *Users*. Aside these key, the symmetric content key, which is used to encrypt the content, is generated by the *CO*. The algorithm for the generation of k_i is not related to our proposed scheme, so we do not delve into it. The following outlines the Setup and Key Generation processes:

A. Setup and Key Generation

The preliminary steps required before executing the schemes are outlined in this subsection.

Setup: While inputting 1^λ , the security parameter, generate the public parameters set, $params$, which is given as follows:

- A prime number q with b bits is selected to create a cyclic additive group. This group, denoted as G , consists of elliptic curves whose order is the large prime q , with P serving as the generator element of G .

- Four hash functions are selected, with n and l representing the lengths of the random bit string utilized by the plaintext and Encrypt algorithm, respectively.

$$\begin{aligned}
 H_1 &: \{0, 1\}^n \times \{0, 1\}^l \times \{0, 1\}^* \times G^2 \rightarrow \mathbb{Z}_q^* \\
 H_2 &: G \rightarrow \{0, 1\}^{n+l} \\
 H_3 &: G \times G \times \{0, 1\}^{n+l} \times G \rightarrow \mathbb{Z}_q^* \\
 H_4 &: \{0, 1\}^* \times \{0, 1\}^* \times G \rightarrow \mathbb{Z}_q^*
 \end{aligned}$$

- The set of public parameters is output as: $params = \{G, q, P, n, l, H_1, H_2, H_3, H_4\}$.

Key Generation: *TTP* generates the public and private key pairs for the *COs*, *SPs*, and *users* using the KeyGenerate algorithm. While inputting the public parameters $params$, this algorithm randomly chooses $sk_i = x_i \in \mathbb{Z}_q^*$ as the private key for User i , and calculates the public key $pk_i = (pk_{i1}, pk_{i2}) = (x_i P, y_i P)$. It then outputs the private key sk_i and the public key (pk_{i1}, pk_{i2}) for User i .

B. Encryption and SC Deployment

The following step involves encrypting m and k_i , which is used to encrypt the m and then deploying the smart contract to the blockchain network.

The algorithms of *CO* consist of functions *encryptContent()*, *encryptContentKey()* and *deployContract()*.

- a) *encryptContent*(m, k_i): *CO* uses its content key, k_i , to encrypt the original content, m , using the *encryptContent()* function and returns an encrypted content C_m .
- b) *encryptContentKey*($k_i, pk_{CO}, id_{CO}, params$): *CO* uses its public key, pk_{CO} , identity, id_{CO} , and *params* to encrypt the content key, k_i , that was used to encrypt the content, using the *encryptContentKey()* function and returns an encrypted key, C .

The encryption algorithm for m is not discussed in the paper, and it is assumed to use a symmetric key. This paper focuses on encrypting k_i , the symmetric key used to encrypt m . The algorithm used for the encryption of the k_i is given below:

Encrypt: While inputting the content key, $k_i \in \{0, 1\}^n$, the public key $pk_{CO} = (pk_{CO1}, pk_{CO2})$, the identity id_{CO} , and *params*, the *CO* does the following:

- a) Select a random ephemeral randomness $c \in \mathbb{Z}_q^*$.
- b) Select an l -bit $\delta \in \{0, 1\}^l$ randomly, and compute $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$, $f = c \cdot r$.
- c) Compute the ciphertext for the content key, $C_1 = rP$, $C_2 = c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1}$, $C_3 = (k_i || \delta) \oplus H_2(fQ_A)$, where $Q_A = P \cdot (pk_{CO1} + pk_{CO2})$.
- d) Select $t \in \mathbb{Z}_q^*$ randomly, and remaining ciphertext is computed, $C_4 = tP$, $C_5 = t + c \cdot r \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1} \cdot H_3(C_1, C_2, C_3, C_4)$.
- e) Original ciphertext for the content key is outputted, $C = (C_1, C_2, C_3, C_4, C_5)$.

After the encryption of k_i , the encrypted content key C is deployed in the ECC-PRE-SC. *CO* executes the *deployContract* functions to deploy the SC and include C and *allowedAddr* in the SC. The overall process of the system reflecting ECC-PRE-SC is shown in Fig. 2.

- a) *deployContract*($C_1, C_2, C_3, C_4, C_5, allowedAddr$): C and *allowedAddr* is set when the ECC-PRE-SC is deployed by the *CO*. The deployed encrypted content key C_1, C_2, C_3, C_4 , and C_5 are kept immutable so that they are read-only. Similarly, hashes of allowed addresses of *SP* are kept in the deployed smart contract to make them indistinguishable from the original addresses of *SP*.

Furthermore, the C_m is retrieved by *SP* from *CO* by calling the *getEncContent* function.

C. User Content Retrieval

User after the valid authentication process can request the content from the *SP*. The function, *reqKeyContent*(pk_{U_i}), requests the re-encrypted content key and encrypted content from *SP* while sending its public key, pk_{U_i} , and receives C_m and C' in return. This process invokes the process of creation of the re-encryption key by *SP* and subsequently the re-encryption process in the SC.

D. Re-encryption Key Generation and Re-encryption of Content Key with ZKP

SP generates the re-encryption key and ECC-PRE-SC performs the re-encryption process. The first process is to generate the re-encryption key, challenge, and proof for the re-encryption process. This is done by the *SP*. *SP* consist of *reqEncContent()*, *reKeyGenerate()*, *getReEncContentKey()*, *computeCommitment()*, *computeChallenge()*, and *computeProof()* functions.

- a) *reqEncContent()*: This function requests encrypted content, C_m , from the *CO* and stores it.
- b) *reKeyGenerate*($c, id_{CO}, pk_{CO}, id_{U_i}, pk_{U_i}$): This function generates a re-encryption key using the ephemeral randomness, c , the identity of the *CO*, id_{CO} , the public key of the *CO*, pk_{CO} , the identity of the User, id_{U_i} , and public key of the User (*U*), pk_{U_i} and returns $rk_{CO \rightarrow U}$.
- c) *getReEncContentKey*($rk_{CO \rightarrow U}$): This function requests the re-encrypted content key from the ECC-PRE-SC, by sending the re-encryption key, $rk_{CO \rightarrow U}$, and receives re-encrypted content key, C' , in return from the ECC-PRE-SC.
- d) *computeCommitment()*: This function computes A , and B , by taking a random value j such that $A = g^j(mod w)$, and $B = h^j(mod w)$, where g and w are generator elements, which belongs to a cyclic group and a large random prime number respectively.
- e) *computeChallenge()*: This functions computes the challenge, α , by concatenating g, h, yy, zz, A and B and using keccak256 hash function.
- f) *computeProof()*: This function computes the proof, γ , such that $\gamma = j - \alpha \cdot (mod w)$.

The algorithm for the generation of the re-encryption key is as follows:

ReKeyGenerate: While inputting the public parameters *params*, ephemeral randomness c , the public key pk_{CO} , the identity id_{CO} of the sender *CO*, the public key $pk_{U_i} = (pk_{U1}, pk_{U2})$, and the identity id_{U_i} of receiver U_i , this algorithm performs as:

- a) Calculate $s = H_4(id_{CO}, id_{U_i}, pk_{U_i})$.
- b) Then, compute $rk_1 = s^{-1} \cdot c \cdot pk_{CO1}$, $rk_2 = s^{-1} \cdot c \cdot pk_{CO2}$, $rk_3 = s^{-1} \cdot (pk_{CO1} + pk_{CO2})$.
- c) The proxy re-encryption key $rk_{A \rightarrow B} = (rk_1, rk_2, rk_3)$ is outputted.

Now the second process is re-encryption. For this *SP* needs to communicate with the ECC-PRE-SC. The re-encryption process is only initiated if the *SP* successfully verifies itself with the SC through a zero-knowledge proof (ZKP) process. After the $rk_{A \rightarrow B}$ is created, ZKP process is initiated with ECC-PRE-SC to verify if the correct *SP* is communicating with ECC-PRE-SC for the re-encryption process. Here, *SP* acts as the prover, embedding a secret message inside a commitment known only to *SP*. *SP* creates a α , and computes the γ , and sends the α and γ to ECC-PRE-SC. ECC-PRE-SC acts as the verifier, receiving the α and γ from *SP*. With the public information and information sent by the *SP*, ECC-PRE-SC, computes a new challenge, α' , and compares it with the α sent by the *SP*. If the challenges are the

same, the verification succeeds, and it proceeds to the re-encryption process. It re-encrypts the C after receiving the $rk_{A \rightarrow B}$ from the SP and returns the C' to the SP . ECC-PRE-SC consist of $reEncrypt()$, $performReEncryption()$ and $verifyProof()$ function.

- $ReEncrypt(i, o, yy, zz, w, \gamma, \alpha, rk_1, rk_2, rk_3)$: This functions, first invokes the $verifyProof()$ function for the Zero Knowledge Proof verification. If correct SP is communicating with the SP $verifyProof()$ returns true and it proceeds further. Then, $performReEncryption()$ is called for the actual re-encryption process. This function re-encrypts the C and returns C' to the SP .
- $verifyProof(i, o, yy, zz, w, \gamma, \alpha)$: This function takes i, o, yy, zz, w, γ and α as input and calculates a new commitment and α' . Then it compares the challenge sent by the SP with the newly derived challenge. If both are equal, it returns true; otherwise, it returns false.
- $performReEncryption(rk_1, rk_2, rk_3)$: The ECC-PRE-SC uses the re-encryption key ($rk_{CO \rightarrow U} = rk_1, rk_2, rk_3$) sent by the SP to re-encrypt the C , which is already in the deployed smart contract, using this function. It then returns a re-encrypted content key C' to the SP .

The re-encrypt algorithm to generate the re-encryption keys is as follows: **Re-Encrypt**: While inputting the ciphertext of the content key C , a re-encryption key $rk_{CO \rightarrow U_i}$, and $params$, re-encryption occurs as follows:

- If $C_5P = C_4 + H_3(C_1, C_2, C_3, C_4)C_2$, then continue; otherwise, output $\perp$.
- Compute $C'_1 = rk_1 \cdot C_1$, $C'_2 = rk_2 \cdot C_1$, $C'_3 = C_3$, $C'_4 = rk_3 \cdot C_1$, and output a new re-encrypted ciphertext of the content key $C' = (id_{CO}, C'_1, C'_2, C'_3, C'_4)$.

Just before the C' , is generated, the increment function of the Count-SC is invoked. The process that occurs in Count-SC is shown in Fig. 4. The Count-SC verifies whether the request to the increment function originates from the address of the ECC-PRE-SC that created it. Once it is verified, Count-SC records the number of times the content is accessed or downloaded. Count-SC, is for counting purposes, which is deployed at the same time ECC-PRE-SC is deployed. Count-SC, as shown in Fig 5, has four functions:

- $constructor()$: It initializes the contract with the owner's address, that is the address of ECC-PRE-SC and the set of allowed addresses, forwarded from ECC-PRE-SC.
- $allowedSender()$: It checks if the caller is allowed to interact with the contract. This function iterates through the list of allowed addresses to determine if the caller is authorized. This function is called privately inside the increment function.
- $increment(sender)$: It first checks if this function is called from ECC-PRE-SC. If it is, it proceeds to the next step; otherwise, it rejects the request. If the check passes, it further verifies if the address passed as the sender is inside the allowed addresses by computing the hash. If this check passes, it increments the count associated with a specific SP 's addresses; otherwise, it rejects the request. Different instances of ECC-PRE-SC are deployed for different content whereas different instances of Count-

SC are deployed for different contents and different SP s. Therefore, for content provided by multiple SP s, this function will increase the count of the specific SP from which the request is received.

- $getCount()$: It returns the count of the content consumed or downloaded with respect to each SP .

E. Decryption Phase

There are two decrypt algorithms, Decrypt, which can be executed by the CO as it has encrypted it, and another ReDecrypt, which can be executed by the end user, where the user uses its own private key to decrypt C' and get k_i . k_i is further used to decrypt the encrypted content.

The user executes the following functions to access the encrypted content and decrypt the content: $reqKeyContent()$, $decryptKey()$, and $decryptContent()$.

- $reqKeyContent()$: This function basically requests C' and C_m with SP .
- $decryptKey(C', id_{CO}, pk_{CO}, id_{U_i}, sk_{U_i}, params)$: U uses the re-encrypted content key, C' , the identity of the CO , id_{CO} , the public key of the CO , pk_{CO} , the identity of User, id_{U_i} , the private key of User, sk_{U_i} , and $params$ to decrypt the re-encrypted content key, C' , using the $decryptKey()$ function, and returns the original content key, k_i , which will be used to decrypt encrypted content.
- $decryptContent(C_m, k_i)$: U uses encrypted content, C_m , and decrypted content key, k_i , to decrypt encrypted content C_m , using the $decryptContent()$ function, and returns the original content m to the user.

The two algorithms are as follows:

Decrypt: While inputting the identity of the CO id_{CO} , the ciphertext of the content key C , the private key of the CO sk_{CO} , and $params$, CO can operates as follows:

- If $C_5P = C_4 + H_3(C_1, C_2, C_3, C_4)C_2$, then proceed; otherwise, output $\perp$.
- Compute $(k_i || \delta) = C_3 \oplus H_2((sk_{CO})C_2)$.
- If $C_2 = c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1}$, where $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$, the algorithm returns k_i as the key to the content. Otherwise, it outputs $\perp$, indicating a decryption failure.

Re-Decrypt: While inputting the identity of the CO , id_{CO} , the re-encrypted content key, C' , the public key of the CO , pk_{CO} , the identity of the U_i , id_{U_i} and private key of the U_i , sk_{U_i} , and public parameter $params$, the user U_i operates as follows:

- Compute $s' = H_4(id_{CO}, id_{U_i}, (sk_{U_i})P)$.
- Compute $(k_i || \delta) = C'_3 \oplus H_2(s'(C'_1 + C'_2))$.
- If $C'_4 = (s')^{-1}rP(pk_{CO1} + pk_{CO2})$, where $r = H_1(m, \delta, id_{CO}, pk_{CO})$, the algorithm returns k_i as the content key. Otherwise, it outputs $\perp$, indicating a decryption failure.

VI. CORRECTNESS AND VALIDATION OF THE SCHEME

A. Correctness of our Proxy Re-encryption Scheme

The proposed ECC based Proxy Re-encryption scheme is correct. There are two defined decryption algorithms, Decrypt

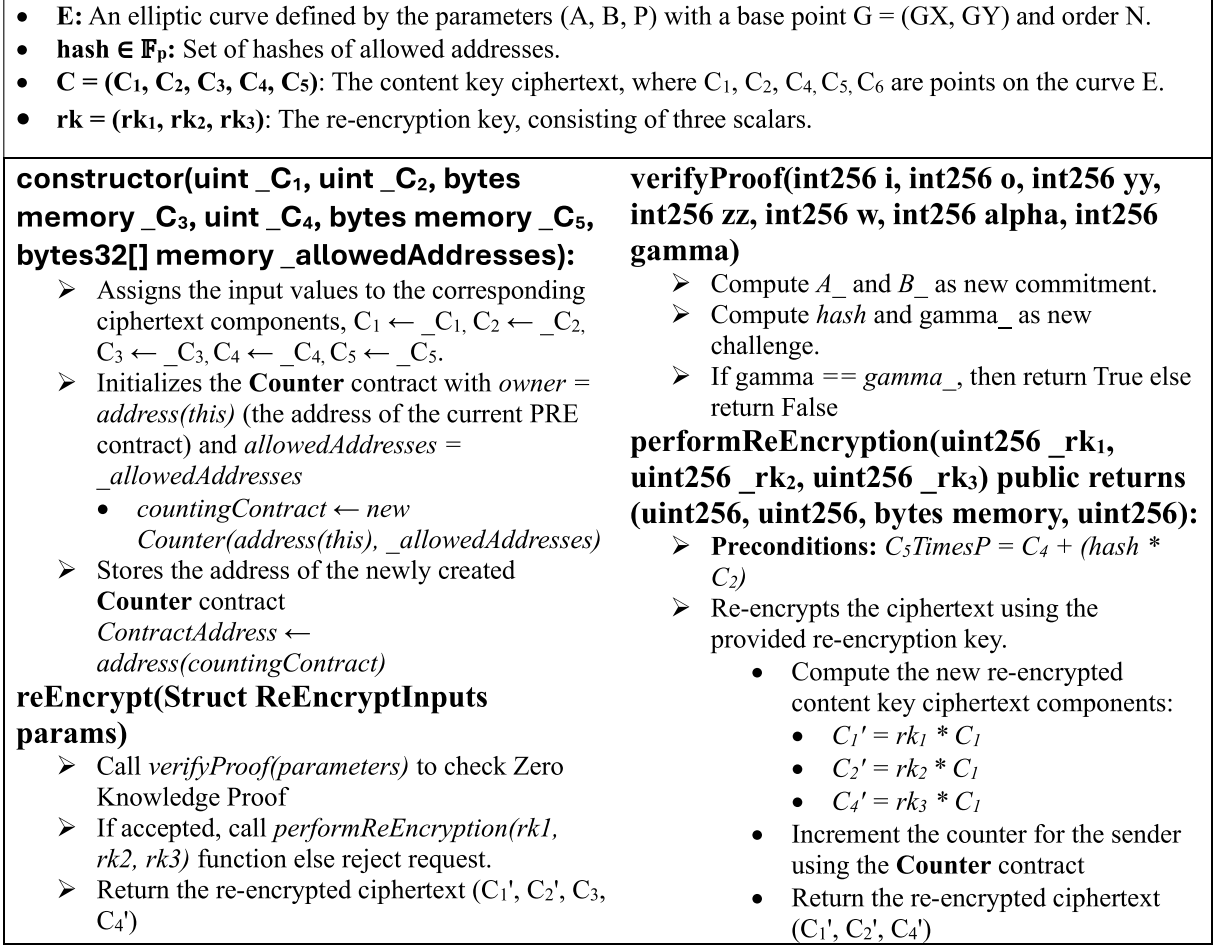


Fig. 3: ECC-PRE-SC Functions

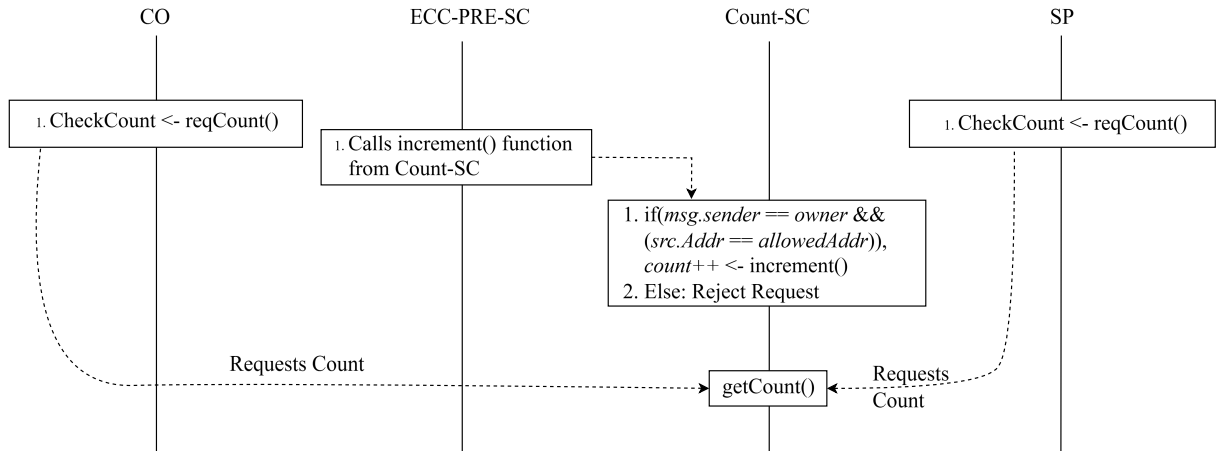


Fig. 4: Count SC Process

and Re-Decrypt. To show the Decrypt correctly gives out the content key, we firstly verify,

$$C_5 P = C_4 + H_3(C_1, C_2, C_3, C_4) C_2$$

Now, we take the left hand side of the equation,

$$C_5 \cdot P = (t + c \cdot r \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1} \cdot H_3(C_1, C_2, C_3, C_4)) \cdot P$$

$$= t \cdot P + c \cdot r \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1} \cdot H_3(C_1, C_2, C_3, C_4) \cdot P$$

$$= C_4 + C_2 \cdot H_3(C_1, C_2, C_3, C_4)$$

After the verification process, the Decrypt algorithm is checked, Here we take,

$$C_3 \oplus H_2(sk_{CO} C_2)$$

$$= C_3 \oplus H_2(sk_{CO} \cdot (c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1}))$$

$$= C_3 \oplus H_2(c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}))$$

$$= C_3 \oplus H_2(f \cdot P \cdot (pk_{CO1} + pk_{CO2}))$$

• A: The set of all possible Ethereum addresses. • H: The set of all possible bytes32 values (hash outputs). • N: The set of natural numbers (including 0). • Bool: The set {TRUE, FALSE} representing boolean values. • owner $\in A$: A constant representing the owner's address. • allowedAddressHashes: $H \rightarrow \text{Bool}$: A function mapping bytes32 hashes to boolean values (TRUE if the hash represents an allowed address, FALSE otherwise). • addressCounts: $A \rightarrow N$: A function mapping addresses to their corresponding counts.	
constructor(address _owner, bytes32[] memory _allowedAddresses): ➤ Preconditions: $_owner \neq 0$ (the zero address) ➤ Initializes the contract with the owner's address and the store the hashes of allowed addresses in a mapping with value as true.	increment(address user) public: ➤ Increments the count associated with the user address if the caller is the owner and the user is in the allowed set A. • If $msg.sender = owner$ and $user \in A$, then $addressCount(user) := addressCount(user) + 1$
allowedSender(address account) internal view returns (bool): ➤ Checks if a hash of given address <i>account</i> is in the allowed address mapping H • Returns <i>true</i> if $\exists a \in H$ such that $keccak256(account) = a$ • Returns <i>false</i> otherwise	getCount(address _user) public view returns (uint) ➤ Returns the current count associated with the user address. set A. • Returns $addressCount(_user)$

Fig. 5: Functions of Counting Smart Contract

$$\begin{aligned}
&= C_3 \oplus H_2(f \cdot Q_A) \\
&= (k_i || \delta) \oplus H_2(f \cdot Q_A) \oplus H_2(f \cdot Q_A) \\
&= (k_i || \delta)
\end{aligned}$$

Hence, the Decrypt algorithm correctly outputs the content key, k_i .

Similar to above for the Re-Decrypt algorithm, we first take

$$\begin{aligned}
s' &= H_4(id_{CO}, id_{U_i}, (sk_{U_i} \cdot P)) \\
&= H_4(id_{CO}, id_{U_i}, (sk_{U_i} \cdot P)) \\
&= H_4(id_{CO}, id_{U_i}, pk_{U1}) \\
&= s
\end{aligned}$$

After above verification, we take the Re-Decrypt algorithm,

$$\begin{aligned}
&C'_3 \oplus H_2(s'(C'_1 + C'_2)) \\
&= C'_3 \oplus H_2(s((rk_1 \cdot C_1) + (rk_2 \cdot C_1))) \\
&= C'_3 \oplus H_2(s((rk_1 \cdot r \cdot P) + (rk_2 \cdot r \cdot P))) \\
&= C'_3 \oplus H_2(s(((s^{-1} \cdot c \cdot pk_{CO1}) \cdot r \cdot P) + ((s^{-1} \cdot c \cdot pk_{CO2}) \cdot r \cdot P))) \\
&= C'_3 \oplus H_2(((c \cdot pk_{CO1}) \cdot r \cdot P) + ((c \cdot pk_{CO2}) \cdot r \cdot P)) \\
&= C'_3 \oplus H_2(c \cdot r \cdot P(pk_{CO1} + pk_{CO2})) \\
&= C_3 \oplus H_2(c \cdot r \cdot Q_A) \\
&= (k_i || \delta) \oplus H_2(f \cdot Q_A) \oplus H_2(f \cdot Q_A) \\
&= (k_i || \delta)
\end{aligned}$$

Here, the delegatee will output the correct content key, k_i .

In summary, the proposed ECC-based PRE scheme is correct.

B. Validation of Smart Contract

We used a collection of security validation tools to check the vulnerability and weak points in our smart contracts. The list of tools that we use to analyze our smart contracts are listed in Table II. Using them, we identified bunch of vulnerabilities with its implication and severity level. We implemented the

TABLE II: Tools used for our Smart Contract Analysis

Tool	Description
semgrep [34]	A static analysis tool that can be used to identify potential security vulnerabilities in code. It uses pattern matching to find common coding errors.
slither-0.10.0 [35]	A static analysis framework for Solidity. It performs analyses on the code to detect vulnerabilities and provides comprehensive reports.
solhint-3.3.8 [36]	A linting tool for Solidity that helps enforce code style guidelines and identify potential errors.
smartcheck [37]	A static analysis tool designed specifically for Solidity. It checks for common security vulnerabilities and code quality issues.
maian [38]	A static analysis tool for Ethereum smart contracts that focuses on detecting security vulnerabilities.
mythril-0.24.7 [39]	A security analysis tool for Ethereum smart contracts. It uses symbolic execution and taint analysis to detect vulnerabilities.
securify [40]	A static analysis tool that formally verifies the security properties of Ethereum smart contracts. It uses a theorem prover to ensure the code adheres to specified security rules.

fix regarding the vulnerabilities. The identified vulnerabilities are as follows:

- 1) **Missing Zero Address Validation:** As identified by Slither tool [35], the PRE contract lacked validation for the zero address in its constructor when receiving the address of the Counter contract. Although the implementation instantiates the Counter contract internally, a potential risk arises if future modifications allow an external Counter contract to be passed as an argument. In such a scenario, a malicious actor could provide the zero address, rendering the access control mecha-

nism ineffective and potentially allowing unauthorized re-encryption operations. This vulnerability has a medium severity due to its conditional nature.

- 2) **Uncached Array Length:** Tools like Semgrep [34] and Slither flag this pattern, as caching the length in a local variable before the loop is a simple optimization that can significantly reduce gas costs, especially for larger arrays. The constructor in the Counter contract iterates through the `allowedAddresses.length` array to verify the length of the allowed addresses. The array length is accessed repeatedly within the loop condition, incurring unnecessary storage reads. This inefficiency, though of low severity, can be easily mitigated for improved gas usage.
- 3) **Unoptimized Boolean Equality:** The `allowedSender` function in the Counter contract employs explicit equality checks against boolean constants (e.g., `if (allowedAddresses[i] == hash)`). While functionally correct, Slither tool identified that this approach is not optimal in terms of gas consumption. Although the severity of this issue is low, addressing it contributes to overall contract efficiency.
- 4) **Immutable State Variables:** The code analysis done by Slither has identified a potential optimization in the PRE contract related to the declaration of state variables. Specifically, several state variables that are assigned values only once in the constructor and never modified afterwards could be declared as `immutable`. Several state variables in the PRE contract are assigned values only once, either during their declaration or within the constructor, and remain constant throughout the contract's lifecycle. These variables, including `c1`, `c1_y`, `c2`, `c2_y`, `c3`, `c4`, `c4_y`, `c5p`, `parity1`, `parity2`, `parity4`, and `hash`, can be declared as `immutable`.

To mitigate the identified vulnerabilities and optimize the contract's performance, we applied the following solutions:

- 1) **Zero Address Check:** Implemented a `require` statement in the Counter contract's constructor to explicitly validate that the `owner` is not the zero address. This simple check effectively prevents the vulnerability associated with a potentially invalid Counter contract address.
- 2) **Array Length Caching:** Cached the length of the `allowedAddresses.length` array in a local variable `allowedAddressesLength` before the loop in the constructor of the Counter contract. This avoids redundant reads from storage during each iteration, improving gas efficiency.
- 3) **Boolean Equality Optimization:** Eliminated explicit comparisons to boolean constants in the `allowedSender` function. The condition `if (allowedAddresses[i] == hash)` was directly replaced with `return allowedAddressHashes[hash]`, simplifying the code and reducing gas costs.
- 4) **Immutable State Variables:** We simply added the `immutable` keyword before the declaration of each of

the identified variables. By adopting this optimization and declaring appropriate state variables as `immutable`, the PRE contract was made more efficient and cost-effective, contributing to a more sustainable and economical blockchain ecosystem.

The tool Maian [38] was used to analyze four different smart contracts (Counter Contract, PRE Contract, FastECMul Library, and ECCLibrary) for three specific vulnerabilities: SUICIDAL, PRODIGAL, and GREEDY. Each contract was subjected to all three checks. The results consistently showed that **none of the contracts were vulnerable to any of the tested conditions**. Specifically, no SUICIDE instructions were found, no instances of CALL/SUICIDE were present, and none of the contracts could receive Ether, eliminating the possibility of lock vulnerabilities. In addition to Maian, the contracts were also analyzed using Mythril [39] and Securify [40], two other security analysis tools. Neither Mythril nor Securify reported any issues or vulnerabilities, further supporting the conclusion that the code is secure.

Furthermore, with the help of the Slither and Solhint [36] tools, we fixed the naming conventions in our code as suggested by the Solidity Style Guidelines, improving code readability and maintainability. We also used the static code analyzer SmartCheck [37] to explore any vulnerabilities in the code. The only suggested changes were to modify the visibility of some variables from private to public, but this was a false positive, as such a change would compromise the security of the contracts.

By implementing these straightforward fixes, the smart contract's security and efficiency can be significantly enhanced. The zero address check eliminates a potential vulnerability, while the boolean equality and array length optimizations reduce gas consumption, contributing to a more cost-effective and robust solution for privacy-preserving re-encryption.

VII. SECURITY ANALYSIS

In this section, we analyse and show that our proposed system model is secure against the threat model defined in Section 4.2.

A. Analyzing multiple re-encrypted keys

Theorem 1. Assume the adversary $\mathcal{A}$ has access to multiple re-encrypted content keys C' on the blockchain and wants to derive the original content key k_i . Additionally, assuming $\mathcal{A}$ has no access to H_2 hash function. The probability that a PPT adversary $\mathcal{A}$ obtains content key k_i by analyzing multiple re-encrypted key is negligible.

Proof. The re-encrypted content key C' kept on the blockchain ledger is generated using a secure elliptic curve cryptography based proxy re-encryption scheme. The security of elliptic curve cryptography is based on the computational hardness of solving the Elliptic Curve Discrete Logarithm Problem (ECDLP). As long as the ECDLP is intractable, it should be computationally infeasible for an adversary to derive the original content key k_i from the re-encrypted ciphertext key C' , even with access to multiple instances.

Let the adversary know the structure of $C' = (C'_1, C'_2, C'_3, C'_4)$, where

$$C'_1 = rk_1 \cdot C_1, C'_2 = rk_2 \cdot C_1, C'_3 = C_3, C'_4 = rk_3 \cdot C_1$$

and $rk_{CO \rightarrow U_i} = (rk_1, rk_2, rk_3)$ is the re-encryption key.

To obtain the original content key k_i from C' , the adversary needs to solve the following equation:

$$\begin{aligned} (k_i || \delta) &= C_3 \oplus H_2(f \cdot Q_A) \\ &\text{or} \\ (k_i || \delta) &= C'_3 \oplus H_2(sk_{CO} \cdot C_2) \\ &\text{or} \\ (k_i || \delta) &= C'_3 \oplus H_2(C'_1 + C'_2) \end{aligned}$$

To solve this equation, the adversary needs to:

- 1) Obtain f , which is $c \cdot r$ where $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$ and c is an ephemeral randomness. Here, to obtain the value of r , $\mathcal{A}$ needs to know the k_i and δ , which are unknown.
- 2) Obtain sk_{CO} which is private to the entity CO only and infeasible to compute with the information that $\mathcal{A}$ has. Additionally, provided that $\mathcal{A}$ has C'_2 , to calculate $C_2 = (rk_2)^{-1} \cdot C'_2$, first, rk_2 is unknown, and second, even if rk_2 were known, it would be computationally infeasible to extract C_2 due to the hardness of the ECDLP.
- 3) As mentioned in the theorem, $\mathcal{A}$ does not have access to the H_2 hash function, so $\mathcal{A}$ cannot compute $(k_i || \delta)$. Furthermore, the value of δ is unknown, making it infeasible to compute k_i .

Provided the above proof, we can conclude that our system is safe against obtaining a content key by analyzing the multiple re-encrypted keys. $\square$

B. Manipulation of Counting in Blockchain

Theorem 2. Consider the Increment function in the Count-SC. The probability that a PPT adversary $\mathcal{A}$ manipulates the count using increment function through the Count-SC is negligible.

Proof. Let Count-SC be the counting smart contract, and ECC-PRE-SC be the re-encryption smart contract with $addr(\text{Count-SC})$ and $addr(\text{ECC-PRE-SC})$ being their respective addresses. During the deployment, an instance of Count-SC is also deployed by keeping $addr(\text{ECC-PRE-SC})$ as the owner of Count-SC. For a CO , SP or an $\mathcal{A}$ to manipulate the count, they would need to consider following scenarios:

- 1) *Tamper Blockchain Ledger:* Modify the Count-SC code, which is immutable on the blockchain and computationally infeasible due to the blockchain's tamper-resistance.
- 2) *Bypass Access Control:* Attempt to directly call the increment function of the Count-SC, regardless of whether they know $addr(\text{Count-SC})$. This is infeasible because the Count-SC is deployed with the $addr(\text{ECC-PRE-SC})$ embedded as its sole owner, enforcing an access control mechanism where only requests originating from $addr(\text{ECC-PRE-SC})$ can trigger the increment function. Any attempt by a valid SP , the CO , or any other adversary to directly call the increment function will be rejected.

- 3) *Perform ZKP for Re-Encryption following Count Increment:* To increment the count legitimately, $\mathcal{A}$ must go through the re-encryption process within ECC-PRE-SC, which requires an $\mathcal{A}$ to pass a Zero-Knowledge Proof (ZKP) verification and also provide re-encryption check before calling the increment function. Only a valid SP can pass the ZKP, leading to the ECC-PRE-SC re-encrypting the content key and subsequently invoking the increment function of Count-SC using its address, $addr(\text{ECC-PRE-SC})$. Without ZKP verification and re-encryption key check, $\mathcal{A}$ request will be rejected.

Since all potential methods for manipulating the count are impractical, the probability of an adversary succeeding in manipulating the count in blockchain is negligible. $\square$

C. Leveraging One Content's Key to Decrypt Another Content

Theorem 3. Assume a malicious user obtains a content key k_1 for encrypted content C_m from a legitimate process and also has obtained an encrypted content (C'_m). The probability that a PPT adversary $\mathcal{A}$ decrypting C'_m with k_1 is negligible.

Proof. Let k_1 be the key used to encrypt content m and produce C_m , and k_2 be the key used to encrypt a different content m' to produce $C_{m'}$. Even if a user obtains k_1 legitimately, they cannot decrypt $C_{m'}$ because:

$$Dec(C_{m'}, k_1) \neq m_2$$

where m_2 is the plaintext of $C_{m'}$

This is due to the key independence property of the encryption scheme, which ensures that different keys are used for different content. Hence, this proves that our system is resistant to leveraging one content's key to decrypt another content. $\square$

D. SP acting as honest but curious entity

Theorem 4. Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary that acts as an honest but curious SP . The probability that $\mathcal{A}$ can obtain the original content key k_i from the transactions it handles is negligible.

Proof. Suppose the CO generates a key pair (sk_{CO}, pk_{CO}) , where sk_{CO} is the private key, and pk_{CO} is the public key. The CO encrypts the content key k_i using pk_{CO} and generates the encrypted content key $C = (C_1, C_2, C_3, C_4, C_5)$. The User (U) also generates a key pair (sk_U, pk_U) , where sk_U is the private key, and pk_U is the public key. The SP acts as an honest but curious adversary $\mathcal{A}$ and generates the re-encryption key $rk_{CO \rightarrow U}$ using the ReKeyGenerate algorithm: $rk_{CO \rightarrow U} = \text{ReKeyGenerate}(c, params, id_{CO}, id_U, pk_{CO}, pk_U)$

Then SP sends $rk_{CO \rightarrow U}$ to the ECC-PRE-SC and receives back a Re-encrypted content key, which it then forwards to the User. To obtain the original content key k_i , the adversary $\mathcal{A}$ needs to satisfy one of the following conditions:

- 1) Compute sk_{CO} or sk_U from the public keys pk_{CO} and pk_U , respectively, and then use the Decrypt or Re-Encrypt algorithms to obtain k_i . However, this would violate the hardness assumption of the ECDLP.
- 2) Compute the value of $c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2})$ from the re-encrypted content key C' , where $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$. However, this would violate the hardness assumption of the CDH problem.

Since both the ECDLP and CDH problems are assumed to be computationally infeasible for the chosen parameters of the elliptic curve, the probability that the adversary $\mathcal{A}$ can obtain the original content key k_i from the transactions it handles is negligible. Therefore, the proposed scheme is secure against Threat Model 4, where the SP acts as an honest but curious entity, attempting to obtain the content key from the transactions it handles. $\square$

E. Security against malicious SP

Theorem 5. *Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary that acts as a valid SP. The probability that $\mathcal{A}$ can impersonate a valid SP or disrupt the ECC-PRE-SC is negligible.*

Proof. For $\mathcal{A}$ to successfully impersonate SP , $\mathcal{A}$ must forge a valid proof and challenge that satisfies the verification function in ECC-PRE-SC. The generation of proofs requires the knowledge of a secret number v and a random number j , which is only known to the valid SP .

Without knowing v and j , the probability of $\mathcal{A}$ generating a valid proof is:

$$Pr[\mathcal{A}(params) = (i, o, yy, zz, A, B, \gamma, \alpha) \mid verifyProof(i, o, yy, zz, A, B, \gamma, \alpha) = true] = \frac{1}{2^{v+j}}$$

where v and j are the length of the secret and random number respectively. As v and j increases, the probability of $\mathcal{A}$ forging a proof becomes negligible.

For $\mathcal{A}$ to disrupt the system, $\mathcal{A}$ must overload ECC-PRE-SC with requests. However, ECC-PRE-SC only responds to requests with valid proofs. Thus, the probability of $\mathcal{A}$ disrupting the system becomes negligible as given by the equation above. The ZKP mechanism ensures that only valid SP s can request the re-encrypted content key and that disrupting the system's functionality is computationally infeasible. $\square$

VIII. IMPLEMENTATION AND EVALUATION

In this section, we discuss the specifics of the implementation and evaluation of our scheme. Specifically, we evaluate the implementation of ECC-PRE-SC, the re-encryption smart contract, and Count-SC, the counting smart contract.

A. Simulation Specifics

The SECP256k1 elliptic curve was used for all elliptic curve point operations. Our program was written and compiled using Solidity 0.7.26 and the elliptic curve arithmetic operations are implemented with the elliptic-curve-solidity and FastEcMul library. We used the Sepolia Blockchain Network and the Alchemy API to connect to the blockchain instance to simulate our scheme. The deployment and interaction were performed

using Python on a computer with an Intel i7-1355U running Windows 11.

B. Performance Analysis

For the following analysis, we let $l = 128$ bits, where l is the length of the key which is used to encrypt the content. In our case, the plaintext data that will be encrypted is a content key, so we let $n = 128, 192$, and 256 bits, respectively, representing various length keys based on key size recommendations for symmetric key lengths for the AES encryption algorithm [41]. As shown in Fig. 6 (a), the size of the message is largely irrelevant in terms of overall gas consumption. The principal driver of gas consumption is the execution of the four elliptic curve scalar multiplication operations (with a 256-bit scalar) and one elliptic curve point addition operation that are all needed to verify and compute the re-encrypted ciphertexts in the Re-Encrypt function. Overall, the ECC-PRE-SC has a consistent gas cost with little variance regardless of the actual message length, so our scheme has great flexibility in the size of the desired re-encrypted message. The counting contract Count-SC consumes far less gas as shown in Fig. 6 (b) because it must only worry about keeping a count. Retrieving the current count for a specific SP costs around 24000 gas in the Sepolia Testnet, whereas as per our verification, the transaction cost for the increment function is not available due to internal transactions between the two contracts. The Count-SC also uses hashes of the approved addresses that are sent by the CO during initialization so that the original addresses are not stored on the blockchain.

During the implementation of the two smart contracts necessary for our scheme, great care was taken to restrict access to the appropriate parties. The CO deploys the ECC-PRE-SC with the necessary parameters $C1, C2, C3, C4, C5$ and a list of the hashed addresses of the SP s who are approved to distribute the content. These values are set as immutable so that after initialization in the constructor they are read-only variables, preventing further manipulation of the values. Then, after receiving a request for content from a User, the SP computes the re-encryption keys from the CO to the User and sends them to the ECC-PRE-SC contract for the re-encryption of the encrypted content key. The re-encryption function only executes if the SP passes the ZKP verification and also the address matches with one provided to the smart contract during initialization by the CO ; otherwise, the SP 's re-encryption request will be rejected outright. Once the ECC-PRE-SC has completed re-encryption, the SP 's specific count value in the Count-SC is incremented, and the re-encrypted content key is returned to the SP and then to the User for decryption to then to access the requested content.

C. Performance Improvement

During the implementation, it was observed that our initial version of the re-encryption scheme consumed a significant amount of gas due to multiple ECC-based operations, including multiplications and additions. The multiplication operation, being the most expensive, prompted us to identify a different library that provides speed-ups for elliptic curve

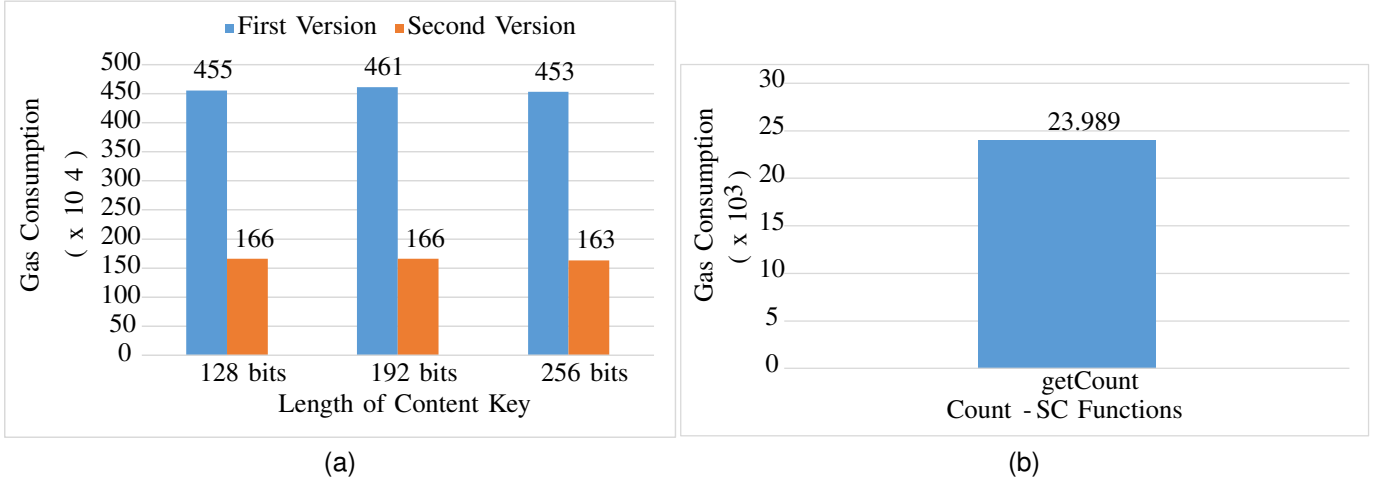


Fig. 6: Gas Consumptions by Smart Contracts (SC). (a) Gas Consumption for ECC-PRE-SC with Different Key Lengths. (b) Gas Consumption for the Count-SC Functions.

multiplication using window Non-Adjacent Form (wNAF) scalar representation and scalar decomposition through endomorphism [42]. Specifically, the gas consumption decreased from 455×10^4 to 166×10^4 for a 128-bit content key length, from 461×10^4 to 166×10^4 for 192 bits, and from 453×10^4 to 163×10^4 for 256 bits. This results in an average gas consumption reduction of approximately 63.84%. The comparison of the gas consumption for our first version of the system and the second improved version has been shown in the Fig. 6 (a).

IX. CONCLUSION

In this paper, we proposed a decentralized solution leveraging smart contracts and blockchain technology to ensure the integrity and accurate tracking of content consumption or downloads. The core contribution involved two key smart contracts: an Elliptic Curve Cryptography-based Proxy Re-Encryption (ECC-PRE) smart contract, ECC-PRE-SC, for secure re-encryption of encrypted content keys, and a Counting smart contract, Count-SC, for maintaining a tamper-proof record of content consumption or downloads. Throughout the deployment and execution, our solution demonstrated consistency, showing that it can re-encrypt messages of widely varying lengths with consistent gas consumption. This architecture enhanced trust and transparency by preventing the manipulation of consumption counts, benefiting COs and SPs to maintain a tamper-free count. By conducting a thorough security analysis and implementation of our system on the Sepolia testnet, we demonstrated its feasibility and security against the various threat models defined in Section IV.B. Multiple static analysis tools were used to identify and fix vulnerabilities in the smart contracts, including missing zero-address validation, inefficient Boolean checks, uncached array lengths, and missed use of immutable variables. These improvements enhanced security and optimized gas usage, resulting in a more robust and efficient solution. After revising from our initial version, we made a significant improvement of approximately 63% in gas consumption, further optimiz-

ing the solution's efficiency. The interactive smart contract framework ensured key confidentiality, prevented unauthorized count changes, and upheld key independence across content. This research offered a resilient alternative to centralized DRM systems, addressing vulnerabilities like single points of failure and lack of transparency.

REFERENCES

- [1] S. Joshi, D. Davidson, and J. Son, "Ensuring integrity in online content usage and download counting with smart contracts," in *Proceedings of the 20th International Conference on Security and Privacy in Communication Networks*, 2024. [Online]. Available: <https://securecomm.eai-conferences.org/2024/accepted-papers/>
- [2] P. Falkowski-Gilski and T. Uhl, "Current trends in consumption of multimedia content using online streaming platforms: A user-centric survey," *Computer Science Review*, vol. 37, p. 100268, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013719302473>
- [3] H. Tabrizchi and M. Kuchaki Rafsanjani, "A survey on security challenges in cloud computing: issues, threats, and solutions," *The Journal of Supercomputing*, vol. 76, no. 12, pp. 9493–9532, Dec. 2020. [Online]. Available: <https://doi.org/10.1007/s11227-020-03213-1>
- [4] A. Tatar, M. D. de Amorim, S. Fdida, and P. Antoniadis, "A survey on predicting the popularity of web content," *Journal of Internet Services and Applications*, vol. 5, no. 1, p. 8, Aug. 2014. [Online]. Available: <https://doi.org/10.1186/s13174-014-0008-y>
- [5] R. Xu, L. Zhang, H. Zhao, and Y. Peng, "Design of network media's digital rights management scheme based on blockchain technology," in *2017 IEEE 13th International Symposium on Autonomous Decentralized System (ISADS)*, 2017, pp. 128–133.
- [6] M. Lee, H. S. Choi, D. Cho, and H. Lee, "Can digital consumption boost physical consumption? the effect of online music streaming on record sales," *Decision Support Systems*, vol. 135, p. 113337, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167923620300920>
- [7] T. Pellegrini, *Digital Rights Management – Technologies, Application Areas, and Governance*. Wiesbaden: Springer Fachmedien Wiesbaden, 2023, pp. 1–17. [Online]. Available: https://doi.org/10.1007/978-3-658-34048-3_79-2
- [8] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [9] A. Foroughi, M. Albin, and S. Gillard, "Digital rights management: a delicate balance between protection and accessibility," *Journal of Information Science*, vol. 28, no. 5, pp. 389–395, 2002. [Online]. Available: <https://doi.org/10.1177/016555150202800504>
- [10] D. KUNDUR and K. KARTHIK, "Video fingerprinting and encryption principles for digital rights management," *Proceedings of the IEEE*, vol. 92, no. 6, pp. 918–932, 2004.

- [11] W. J. Buchanan, "Nizk (non-interactive zero knowledge) proofs of discrete - log equality with powershell," 2025. [Online]. Available: <https://asecuritysite.com/powershell/chaum>
- [12] X. Xiao, Y. Zhang, L. Y. Zhang, Z. Hua, Z. Liu, and J. Huang, "Faircms: Cloud media sharing with fair copyright protection," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 5, pp. 6192–6209, 2024.
- [13] T. Feng, D. Wang, and R. Gong, "A blockchain-based efficient and verifiable attribute-based proxy re-encryption cloud sharing scheme," *Information*, vol. 14, no. 5, 2023. [Online]. Available: <https://www.mdpi.com/2078-2489/14/5/281>
- [14] L. Yan, H. Qin, K. Yang, H. Xie, X. A. Wang, and S. Liu, "Pairing-free certificate-based proxy re-encryption plus scheme for secure cloud data sharing," *Electronics*, vol. 13, no. 3, 2024. [Online]. Available: <https://www.mdpi.com/2079-9292/13/3/534>
- [15] Z. Zhang and L. Zhao, "A design of digital rights management mechanism based on blockchain technology," in *Blockchain - ICBC 2018*, S. Chen, H. Wang, and L.-J. Zhang, Eds. Cham: Springer International Publishing, 2018, pp. 32–46.
- [16] A. Garba, A. D. Dwivedi, M. Kamal, G. Srivastava, M. Tariq, M. A. Hasan, and Z. Chen, "A digital rights management system based on a scalable blockchain," *Peer-to-Peer Networking and Applications*, vol. 14, no. 5, pp. 2665–2680, Sep. 2021. [Online]. Available: <https://doi.org/10.1007/s12083-020-01023-z>
- [17] J. Gao, H. Yu, X. Zhu, and X. Li, "Blockchain-based digital rights management scheme via multiauthority ciphertext-policy attribute-based encryption and proxy re-encryption," *IEEE Systems Journal*, vol. 15, no. 4, pp. 5233–5244, 2021.
- [18] V. Thangam and K. Chandrasekaran, "Elliptic curve based proxy re-encryption," in *Proceedings of the Second International Conference on Information and Communication Technology for Competitive Strategies*, ser. ICTCS '16. New York, NY, USA: Association for Computing Machinery, 2016. [Online]. Available: <https://doi.org/10.1145/2905055.2905337>
- [19] G. Ateniese, K. Fu, M. Green, and S. Hohenberger, "Improved proxy re-encryption schemes with applications to secure distributed storage," *ACM Trans. Inf. Syst. Secur.*, vol. 9, no. 1, p. 1–30, Feb. 2006. [Online]. Available: <https://doi.org/10.1145/1127345.1127346>
- [20] K. O.-B. O. Agyekum, Q. Xia, E. B. Sifah, C. N. A. Cobblah, H. Xia, and J. Gao, "A proxy re-encryption approach to secure data sharing in the internet of things based on blockchain," *IEEE Systems Journal*, vol. 16, no. 1, pp. 1685–1696, 2022.
- [21] I. Keshta, Y. Aoudni, M. Sandhu, A. Singh, P. A. Xalikovich, A. Rizwan, M. Soni, and S. Lalar, "Blockchain aware proxy re-encryption algorithm-based data sharing scheme," *Physical Communication*, vol. 58, p. 102048, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1874490723000514>
- [22] P. Shabisha, A. Braeken, A. Touhafi, and K. Steenhaut, "Elliptic curve qu-vanstone based signcryption schemes with proxy re-encryption for secure cloud data storage," in *Cloud Computing and Big Data: Technologies, Applications and Security*, M. Zbakh, M. Essaïdi, P. Manneback, and C. Rong, Eds. Cham: Springer International Publishing, 2019, pp. 1–18.
- [23] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Advances in Cryptology - CRYPTO 2017*, J. Katz and H. Shacham, Eds. Cham: Springer International Publishing, 2017, pp. 357–388.
- [24] K. Wüst and A. Gervais, "Do you need a blockchain?" in *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*, 2018, pp. 45–54.
- [25] N. Radziwill, "Blockchain revolution: How the technology behind bitcoin is changing money, business, and the world," *The Quality Management Journal*, vol. 25, no. 1, pp. 64–65, 2018, name - American Society for Quality; Penguin Random House; Copyright - Copyright Taylor & Francis Ltd. 2018; Last updated - 2023-12-04; SubjectsTermNotLitGenreText - New York; Ecuador; United States-US. [Online]. Available: <https://www.ezproxy.library.unlv.edu/login?url=https%3A%2F%2Fwww.proquest.com%2Fscholarly-journals%2Fblockchain-revolution-how-technology-behind%2Fdocview%2F2114612269%2Fse-2%3Faccountid%3D3611>
- [26] S. Ølnes, "Beyond bitcoin enabling smart government using blockchain technology," in *Electronic Government*, H. J. Scholl, O. Glassey, M. Janssen, B. Klievink, I. Lindgren, P. Parycek, E. Tambouris, M. A. Wimmer, T. Janowski, and D. Sá Soares, Eds. Cham: Springer International Publishing, 2016, pp. 253–264.
- [27] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," pp. 1–32, 2014. [Online]. Available: <https://cryptodeep.ru/doc/paper.pdf>
- [28] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3190508.3190538>
- [29] M. Likhava, G. Losa, D. Mazières, G. Hoare, N. Barry, E. Gafni, J. Jove, R. Malinowsky, and J. McCaleb, "Fast and secure global payments with stellar," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, ser. SOSP '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 80–96. [Online]. Available: <https://doi.org/10.1145/3341301.3359636>
- [30] A. Manzoor, A. Braeken, S. S. Kanhere, M. Ylianttila, and M. Liyanage, "Proxy re-encryption enabled secure and anonymous iot data sharing platform based on blockchain," *Journal of Network and Computer Applications*, vol. 176, p. 102917, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804520303763>
- [31] M. Bartoletti and L. Pompianu, "An empirical analysis of smart contracts: Platforms, applications, and design patterns," in *Financial Cryptography and Data Security*, M. Brenner, K. Rohloff, J. Bonneau, A. Miller, P. Y. Ryan, V. Teague, A. Bracciali, M. Sala, F. Pintore, and M. Jakobsson, Eds. Cham: Springer International Publishing, 2017, pp. 494–509.
- [32] J.-C. Cheng, N.-Y. Lee, C. Chi, and Y.-H. Chen, "Blockchain and smart contract for digital certificate," in *2018 IEEE International Conference on Applied System Invention (ICASI)*, 2018, pp. 1046–1051.
- [33] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, "The keccak reference," 2011, <https://keccak.team/files/Keccak-reference-3.0.pdf>.
- [34] "Decurity/semgrep-smart-contracts: Semgrep rules for smart contracts based on defi exploits," 2024. [Online]. Available: <https://github.com/Decurity/semgrep-smart-contracts?tab=readme-ov-file>
- [35] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.
- [36] "Solhint," 2017. [Online]. Available: <https://github.com/protofire/solhint>
- [37] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "Smartcheck: static analysis of ethereum smart contracts," in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, ser. WETSEB '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 9–16. [Online]. Available: <https://doi.org/10.1145/3194113.3194115>
- [38] I. Nikolić, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, "Finding the greedy, prodigal, and suicidal contracts at scale," in *Proceedings of the 34th Annual Computer Security Applications Conference*, ser. ACSAC '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 653–663. [Online]. Available: <https://doi.org/10.1145/3274694.3274743>
- [39] "Smashing ethereum smart contracts for fun and real profit," 2018. [Online]. Available: <https://github.com/muellerberndt/smashing-smart-contracts/blob/master/smashing-smart-contracts-1of1.pdf>
- [40] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 67–82. [Online]. Available: <https://doi.org/10.1145/3243734.3243780>
- [41] E. Barker and A. Roginsky, "Transitioning the use of cryptographic algorithms and key lengths," 2019. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-131Ar2>
- [42] "elliptic-curve-solidity," 2019. [Online]. Available: <https://github.com/witnet/elliptic-curve-solidity/tree/master>